FlashAttention

涉及主要技术：

1. Tilling - 优化KQV矩阵乘法
2. Re-computation

核心背景：

传统的注意力机制在处理长序列时面临处理速度慢和内存占用大的问题

- 原因：因为标准注意力机制的时间和空间复杂度为序列长度的二次方（N^2）

近似注意力机制降低计算复杂度：

1. 问题：通过牺牲模型质量来实现，且实际执行速度（wall-clock）并没有明显提升
    a. Wall-clock：从程序运行开始到结束的实际时间
2. 传统近似注意力机制浅讲：
    a. 稀疏注意力 - Sparse Attention
        i. 核心思想：不是每个token都需要关注其余所有的tokens，往往一个token只与附近的词或少量关键词相关，因此强制让注意力矩阵变稀疏（即大部分值为0），只关注重要的部分
        ii. 典型实现：
            1. Longformer：结合局部窗口注意力（只看前后相邻的 token）和全局注意力（选少数关键 token 看全局）
            2. Sliding Window Attention (SWA)：如 Mistral、Llama 3 中使用的方法，每个 token 只关注最近的一个固定窗口；虽然当前的 token 无法直接看到那些已经滑出窗口的古老信息，但这些古老信息并没有完全消失，而是通过中间层级的 Token 像“接力棒”一样，一步步传递到了当前时刻
3. 运行速度没有明显提升的原因：尽管计算需求已经被降低到seq len的线性或非线性水平，但是因为只关注于FLOPs的减少，缺少了对注意力算法的 IO-感知，即考虑GPU不同层级存储的读写
    a. 1 FLOP = 一次浮点加法或乘法操作
    b. 在现代GPU中计算速度已经远远大于显存带宽，所以大多数操作实际受限于内存访问速度

标准注意力机制：

1. 输入矩阵$Q, K, V \in R^{N\times d}$，N为序列长度，d为注意力头维度
2. 计算步骤：
    a. 计算相似度：$S = \frac{1}{\sqrt{d_k}} QK^T \in R^{N\times N}$, $S_{ij} = \frac{1}{\sqrt{d_k}} \sum_{l=1}^{d_k} Q_{il} K_{jl}$
    b. 计算注意力权重：$P = softmax(S) \in R^{N\times N}$

i. $softmax$ 对S进行逐行归一化，及对矩阵的每一行独立的进行softmax操作，每一行所有元素之和为1

ii. 基础公式：$P_{ij} = \frac{e^{S_{ij}}}{\sum_{k=1}^{N} e^{S_{ik}}}$ ，$\sum_{k=1}^{N} e^{S_{ik}}$为归一化因子

iii. 数值稳定性公式：$P_{ij} = \frac{e^{S_{ij} - \max_k (S_{ik})}}{\sum_{k=1}^{N} e^{S_{ik} - \max_k (S_{ik})}}$ ，$\sum_{k=1}^{N} e^{S_{ik} - \max_k (S_{ik})}$为归一化因子

iv. 归一化因子对第i行来说，为一个标量

c. 最终输出：$O = PV = \sum_{j=1}^{N} P_{ij} V_{jk} \in R^{N \times d}$

3. 瓶颈：
   a. 由于会将S和P完整存储在HBM中，导致$O(N^2)$的内存占用
   b. softmax等操作在GPU上的瓶颈在于内存访问速度，而非计算能力，由于通常N远远大于d，频繁读写S和P的大矩阵会产生巨大IO开销，导致运行速度缓慢
   c. 逐元素操作会进一步增加HBM读写次数，比如对S的mask操作和对P的dropout操作（可以通过融合算子解决）
4. 执行算法：

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

5. 对于多头注意力机制，相同的操作并行的在多个头计算，沿批次维度并进行处理
6. 反向传播：已知上游梯度：$d_O = \frac{\partial L}{\partial O}$ (标量损失L对O的梯度) $\in R^{N \times d}$
   a. 目标：计算d_Q; d_K; d_V
   b. 推导顺序：O -> P, V -> S -> Q,K
   c. 计算$O = PV$的梯度
      i. $d_P = \frac{\partial L}{\partial P} = d_O V^T \in R^{N \times N}$
         - 证明：根据链式法则：
           $\frac{\partial L}{\partial P_{ij}} = \sum_{k=1}^{d} \frac{\partial L}{\partial O_{ik}} \frac{\partial O_{ik}}{\partial P_{ij}}$ (dv是v的维度；$\frac{\partial O_{ik}}{\partial P_{ij}} = V_{jk}$；$\frac{\partial L}{\partial O_{ik}} = d_{O,ik}$) $= \sum_{k=1}^{d} d_{O,ik} V_{jk}$
           - 矩阵维度：$(d_O V^T)_{ij} = \sum_k d_{O,ik} (V^T)_{kj} = \sum_k d_{O,ik} V_{jk}$
      ii. $d_V = \frac{\partial L}{\partial V} = P^T d_O \in R^{N \times d}$
      iii. $d_S = (diag(P) - PP^T) d_P \in R^{N \times N}$
      iv. $d_Q = d_S K \in R^{N \times d}$
      v. $d_K = Q d_S^{\ T} \in R^{N \times d}$

FlashAttention

简介：

是一种具备IO-感知的精确注意力算法，通过使用tiling技术降低访问HBM（Global Memory）的次数

成果简述：

1. Bert-large(seq len 512) 提速 15%
2. GPT2(seq len 1K) 提速 3 倍

实现方式：

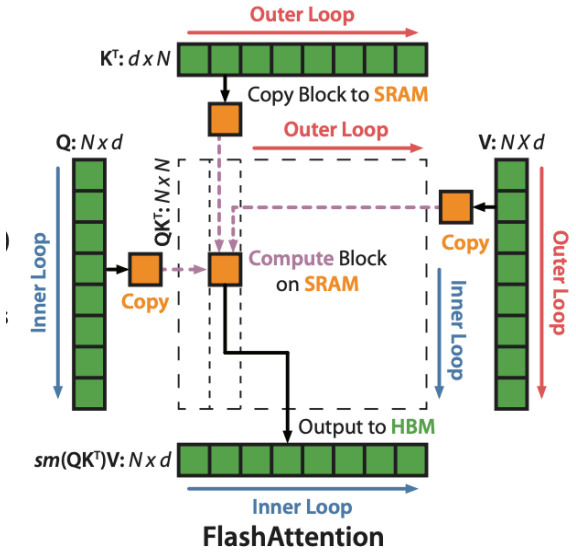
通过cuda实现对内存访问的细粒度控制和算子融合

核心目标：避免在HBM中读写 NxN 的注意力矩阵

1. 为实现该核心目标需要：
    a. 在不访问完整输入的情况下计算softmax
    b. 在反向传播时不存储巨大的中间注意力矩阵（即 softmax(KV) 的结果矩阵）
2. 解决方案：
    a. Tiling
    b. Recomputation

Tiling：

1. 通过重构注意力计算，将输入分解为blocks，并多次传递，递归增量的计算softmax
2. Tiling过程（根据SRAM大小动态计算的非对称分块策略）：外圈切K/V，内圈切Q



    a. 两个关键维度：
        i. B_r：Block Row Size，对应Q的分块，每次只处理B_r个token的Query 向量，通常较小（例如：64或128），以节省空间用于存储中间变量（m：当前已见的数据最大值；l：当前已见数据的归一化因子）

ii. B_c：Block Column Size，对应K 和 V的分块，每次加载B_c个token的KV对进入SRAM计算，通常较大（例如：64、128甚至256），目的尽可能填满SRAM，减少循环次数

iii. B_r和B_c不一定一样大，都不等于序列长度N，主要取决于头维度和SRAM

b. 具体分块步骤：假设$B_r = 64$; $B_c = 128$; 序列长度$N = 4096$; $d = 128$

i. 外层循环：Q（4096×128）沿着序列维度切成64个块（$T_r = 4096/64 = 64$）

1. Q_i: (B_r, d)
2. 每个SM负责计算这64个query对应的O_i

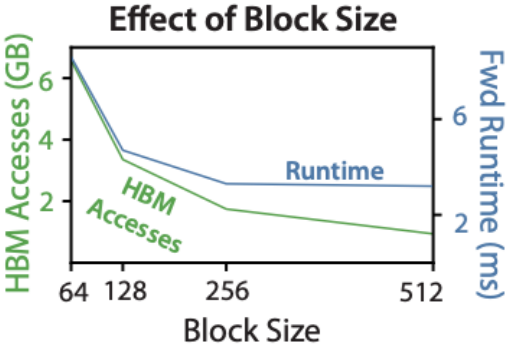
ii. 内存循环：对于每一个Q_i，需要遍历所有的K和V，沿着序列维度切成32个块

1. K_j, V_j: (B_c, d)
2. 局部注意力计算：$S_{ij} = Q_i \times K_j^T$; S_{ij}: (B_r, B_c)

Recomputation：

1. 由下图可见，GPT2中HBM访问为影响运行时间的主要因素

Attention	Standard	FlashAttention
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3



2. 在前向传播中仅只存储最终输出 O 和归一化统计量 (m,ℓ)，在反向传播过程中快速重新计算注意力矩阵 S 和 P
3. 尽管增加了计算量，但是由于降低了HBM访问次数，速度还是更快和更少的内存占用

Online Softmax：

1. 解决的问题：如何在不知道整行数据的情况下，计算出正确、数值稳定的softmax值
   a. 原因：因为序列长度太长，无法一次性放入高速缓存（SRAM）
   b. 整行数据：注意力分数矩阵S的一行，第i行对应第i个query与序列中所有Key token的注意力分数
2. 数学推导：将一行数据x分块为$x^{(1)}$和$x^{(2)}$
   a. 对于任意已处理的分块数据，维护两个标量：
      i. M：当前已见的数据最大值
      ii. L：当前已见数据的归一化因子
      iii. o：$\sum p_i v_i$

b. 合并步骤：假设已经处理完成$x^{(1)}$，得到了局部统计量(m_1, l_1)

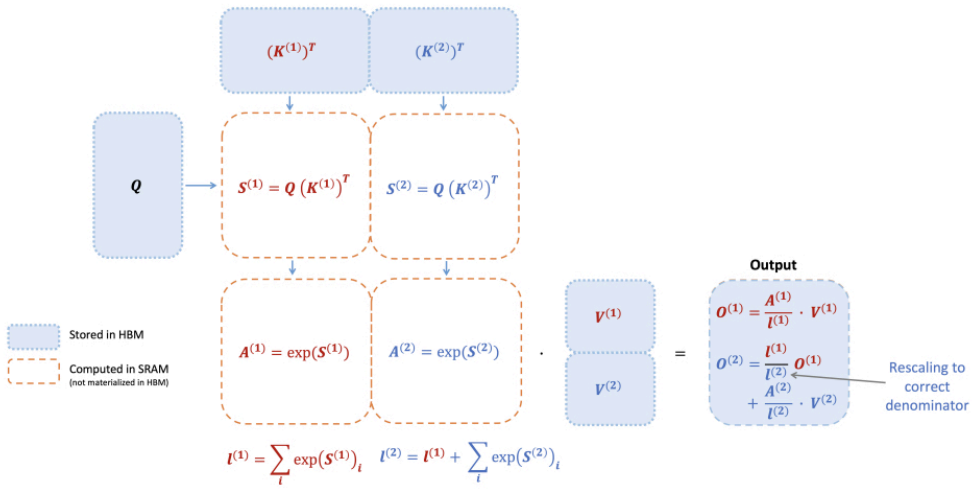
i. 加载$x^{(2)}$计算(m_2, l_2)

1. $m_2 = max(x^{(2)})$

2. $l_2 = \sum_{x \in x^{(2)}} e^{x-m_2}$

ii. 计算合并后的全局统计量(m, l)

1. Step 1: 更新全局最大值 $m_{new} = max(m_1, m_2)$
2. Step 2: 更新全局归一化因子
   a. Tips：$e^{x-m_{new}} = e^{x-m_{old}} * e^{m_{old}-m_{new}}$
   b. $l'_1 = l_1 * e^{m1-m_{new}}$
   c. $l'_2 = l_2 * e^{m2-m_{new}}$（如果$m_2$=$m_{new}$，则$l'_2 = l_2$）
   d. 所以：$l_{new} = l_1 * e^{m1-m_{new}} + l_2 * e^{m2-m_{new}}$
3. Step 3: 修正之前的输出（Rescale）
   a. $P^{(1)}_{old} = \frac{e^{x-m_1}}{l_1}$
   b. $P^{(1)}_{new} = \frac{e^{x-m_{new}}}{l_{new}} = \frac{e^{x-m_1} * e^{m_1-m_{new}}}{l_{new}} = P^{(1)}_{old} * \frac{l_1}{l_{new}} * e^{m_1-m_{new}}$
   c. 所以最终输出修改为：$o_1' = o_1 * \frac{l_1}{l_{new}} * e^{m_1-m_{new}}$
4. Step 4: 合并当前块的输出：
   a. $o_1' = o_1 * \frac{l_1}{l_{new}} * e^{m_1-m_{new}}$
   b. $o_{new} = o_1 * \frac{l_1}{l_{new}} * e^{m_1-m_{new}} + o_2 * \frac{l_2}{l_{new}} * e^{m_2-m_{new}}$
   c. 代码中常用实现：由于除法运算在GPU上实际很慢；所以使用未归一化的加权和，最后再除以l_{new}
      i. $o_{new} = o_1 * e^{m_1-m_{new}} + o_2 * e^{m_2-m_{new}}$



Algorithm 1 FlashAttention

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

Set block sizes $B_c = \left\lceil \frac{M}{4d} \right\rceil, B_r = \min\left(\left\lceil \frac{M}{4d} \right\rceil, d\right)$. tiling切割尺寸计算

Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each. QKV切为blocks

Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.

for $1 \le j \le T_c$ **do** 循环所有K,V

  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.

  for $1 \le i \le T_r$ **do** 循环所有Q

    Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.

    On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute $\tilde{m}_{ij} = \mathrm{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \mathrm{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$. 计算softmax及统计量

    On chip, compute $m_i^{\mathrm{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\mathrm{new}} = e^{m_i - m_i^{\mathrm{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\mathrm{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$. 更新统计量

    Write $\mathbf{O}_i \leftarrow \mathrm{diag}(\ell_i^{\mathrm{new}})^{-1}(\mathrm{diag}(\ell_i) e^{m_i - m_i^{\mathrm{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\mathrm{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM. 合并结果

    Write $\ell_i \leftarrow \ell_i^{\mathrm{new}}$, $m_i \leftarrow m_i^{\mathrm{new}}$ to HBM.

  end for

end for

Return $\mathbf{O}$.

3. 前向和反向传播都只需要O(n)的额外缓存

IO 复杂度对比：

标准注意力机制 IO 复杂度：$O(Nd + N^2)$，Flashattention注意力机制 IO 复杂度：$O(N^2 d/M)$

1. N： 序列长度
2. d： 每个注意力头的维度
3. M： GPU 片上高速缓存（SRAM）的大小
4. Nd部分：这是读取QKV和最后写入输出O的代价，无法避免
5. N^2部分：前向传播时需要将NxN的中间结果矩阵存入显存，反向传播又需要读取，显存访问次数与N^2成正比
6. Flashattention使用tiling将NxN的大矩阵切分为多个blocks，每次只加载一个block带大小为M的SRAM中，所以需要访问内存次数为$N^2 d/M$

总结：

FlashAttention 的运行时间虽然随序列长度呈二次方增长，但速度明显快于所有精确注意力的基准测试。FlashAttention 的内存需求随序列长度呈线性增长，而非标准注意力的二次方增长；相比标准实现，它最高能节省 20 倍 的内存

FlashAttention2

FlashAttention 1的不足：

FlashAttention的速度远远没有达到将优化GEMM操作（矩阵乘法），仅能达到理论最大FLOPs/s的25-40%

1. 原因：因为GPU不同SMs和warps工作分配不合理，导致的SM利用率低或者共享内存不必要的读写

2. 优化GEMM操作可以达到80-90%的理论设备吞吐

FlashAttention 2通过更优的工作分配解决上面的问题：

解决方案：

1. 通过调整算法，减少非矩阵乘法的FLOPs
2. 并行：
    a. 即使对于单个注意力头，也应在不同的SM中并行的实现注意力计算，提高线程利用率
    b. 在原有的batch和头维度并行的基础上，增加seq维度并行，在长序列（通常batch size较小）的情况下，会增加利用率
3. 在每个SM中，将工作分配到不同的warps（以warp为分配单位）减少通过共享内存的通信
    a. 同一个warp内可以通过fast shuffle指令直接交换彼此寄存器实现快速通信

成果：

1. 比起FlashAttention提速2倍
2. 更接近GEMM操作的效率
3. 在A100上达到理论最大FLOPs/s的50-73%，72%的模型FLOPs利用率

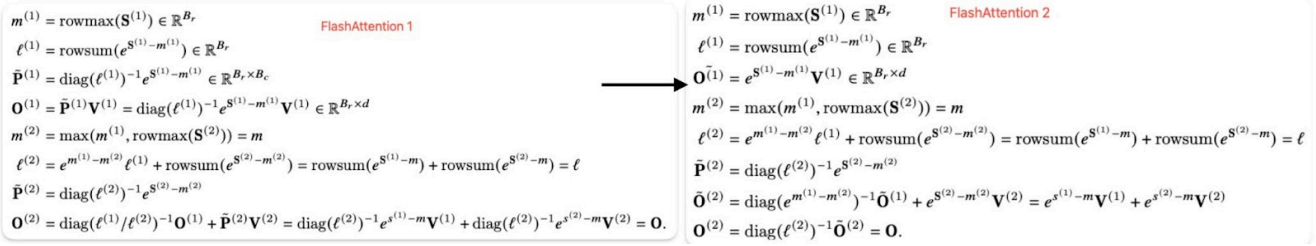
调整算法，减少非矩阵乘法的FLOPs：

优化原因：

1. 尽管非矩阵乘法的FLOPs只占全部FLOPs的一小部分，但是由于GPU有专门用于矩阵乘法的运算单元，所以非矩阵乘法实际的执行时间很久
2. 矩阵乘法的吞吐量是非矩阵乘法吞吐量的16倍以上

调整算法

1. 不需要每一次都对两个项除以l_{new}更新（$o_{new} = o_1 * \frac{l_1}{l_{new}} * e^{m_1 - m_{new}} + o_2 * \frac{l_2}{l_{new}} * e^{m_2 - m_{new}}$）；可以通过每次保存的是为rescale的O和保存统计量l在每轮村换末尾对使用最新的l更新最后的O



2. 不需要为了反向传播保存m和l，可以直接保存$L(j) = m^{(j)} + log(l^{(j)})$
    - tips：$a/b = \exp(\ln(a) - \ln(b))$
    - 证明：

$$P_{ij} = \frac{\exp(S_{ij} - m_i)}{\ell_i}$$

$$P_{ij} = \exp\left(\ln\left(\frac{\exp(S_{ij} - m_i)}{\ell_i}\right)\right)$$

$$P_{ij} = \exp\left((S_{ij} - m_i) - \log(\ell_i)\right)$$

$$P_{ij} = \exp\left(S_{ij} - (m_i + \log(\ell_i))\right)$$

Algorithm 1 FlashAttention-2 forward pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, block sizes B_c, B_r.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.

Divide the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, and divide the logsumexp L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.

for $1 \le i \le T_r$ **do**

  Load $\mathbf{Q}_i$ from HBM to on-chip SRAM.

  On chip, initialize $\mathbf{O}_i^{(0)} = (0)_{B_r \times d} \in \mathbb{R}^{B_r \times d}, \ell_i^{(0)} = (0)_{B_r} \in \mathbb{R}^{B_r}, m_i^{(0)} = (-\infty)_{B_r} \in \mathbb{R}^{B_r}$.

  for $1 \le j \le T_c$ **do**

    Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.

    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute $m_i^{(j)} = \max(m_i^{(j-1)}, \mathrm{rowmax}(\mathbf{S}_i^{(j)})) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_i^{(j)} = \exp(\mathbf{S}_i^{(j)} - m_i^{(j)}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\ell_i^{(j)} = e^{m_i^{j-1} - m_i^{(j)}} \ell_i^{(j-1)} + \mathrm{rowsum}(\tilde{\mathbf{P}}_i^{(j)}) \in \mathbb{R}^{B_r}$.

    On chip, compute $\mathbf{O}_i^{(j)} = \mathrm{diag}(e^{m_i^{(j-1)} - m_i^{(j)}})^{-1} \mathbf{O}_i^{(j-1)} + \tilde{\mathbf{P}}_i^{(j)} \mathbf{V}_j$.

  end for

  On chip, compute $\mathbf{O}_i = \mathrm{diag}(\ell_i^{(T_c)})^{-1} \mathbf{O}_i^{(T_c)}$.

  On chip, compute $L_i = m_i^{(T_c)} + \log(\ell_i^{(T_c)})$.

  Write $\mathbf{O}_i$ to HBM as the i-th block of $\mathbf{O}$.

  Write L_i to HBM as the i-th block of L.

end for

Return the output $\mathbf{O}$ and the logsumexp L.

因果掩码：

1. 确保当前位置只能“看到”之前的信息，即当j>i时，将S_{ij}设为-∞
2. FlashAttention 和 FlashAttention-2 利用Tiling计算的特性：
   a. 由于因果掩码矩阵是一个下三角矩阵，在分块处理时，会有约一半的块完全落在上三角区域（即这些块内所有的列索引 j 都大于行索引 i），可以直接识别并跳过这些块的加载与计算；使速度比不带掩码的注意力机制提升了约 1.7-1.8 倍
   b. 只有那些正好跨越对角线的块（即部分 j>i 且部分 j≤i），才需要进行逐元素的掩码处理；对于每一行，通常只有 1 个块需要执行复杂的掩码逻辑，其余块要么直接跳过，要么按无掩码处理

反向传播：

Algorithm 2 FlashAttention-2 Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vector $L \in \mathbb{R}^N$ in HBM, block sizes B_c, B_r.

Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.

Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, and divide L into T_r blocks $L_i, \dots, L_{T_r}$ of size B_r each.

Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Divide $\mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.

Compute $D = \mathrm{rowsum}(\mathbf{dO} \circ \mathbf{O}) \in \mathbb{R}^d$ (pointwise multiply), write D to HBM and divide it into T_r blocks $D_1, \dots, D_{T_r}$ of size B_r each.

for $1 \le j \le T_c$ **do**

  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.

  Initialize $\mathbf{dK}_j = (0)_{B_c \times d}, \mathbf{dV}_j = (0)_{B_c \times d}$ on SRAM.

  for $1 \le i \le T_r$ **do**

    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, L_i, D_i$ from HBM to on-chip SRAM.

    On chip, compute $\mathbf{S}_i^{(j)} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute $\mathbf{P}_i^{(j)} = \exp(\mathbf{S}_{ij} - L_i) \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute $\mathbf{dV}_j \leftarrow \mathbf{dV}_j + (\mathbf{P}_i^{(j)})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.

    On chip, compute $\mathbf{dP}_i^{(j)} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.

    On chip, compute $\mathbf{dS}_i^{(j)} = \mathbf{P}_i^{(j)} \circ (\mathbf{dP}_i^{(j)} - D_i) \in \mathbb{R}^{B_r \times B_c}$.

    Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.

    On chip, compute $\mathbf{dK}_j \leftarrow \mathbf{dK}_j + \mathbf{dS}_i^{(j)\top} \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.

  end for

  Write $\mathbf{dK}_j, \mathbf{dV}_j$ to HBM.

end for

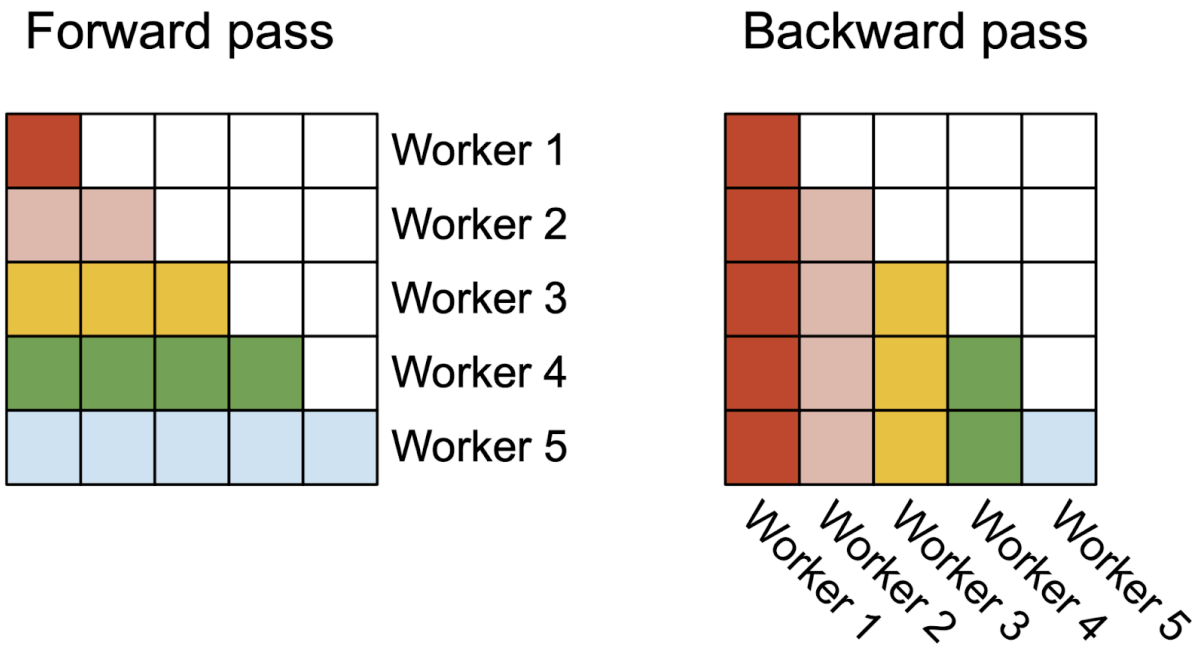
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

并行：

1. FlashAttention 1 的并行为batch size和注意力头数量，一个SM处理一个注意力头的处理，总并行数量为batch size x 注意力头数量； Batch Size × Heads 的数值较小，就无法填满所有的 SM，导致硬件资源闲置
2. FlashAttention 2 在原有基础上增加了seq维度的并行，解决长序列（通常batch size和注意力头数量较小）的GPU利用低的问题
3. 前向传播：将FlashAttention 1的内外层顺序改为外层Br内层Bc，由于外层循环（遍历行块）是“易并行化，通过使用不同的SM并行处理，且每个SM间不需要互相通信，同时在batch size和注意力头数量也并行
    a. Softmax 操作是逐行进行的，这意味着输出矩阵 O 的每一行只取决于该行对应的 Query 以及所有的 Key 和 Value。不同行之间在计算过程中没有任何数据耦合，不需要互相等待或交换中间结果
4. 反向传播：根据反向传播发现，共享的计算仅在不同的column blocks更新dQ，使用原子加法来确保 dQ 的更新是正确且安全的

Load $\mathbf{dQ}_i$ from HBM to SRAM, then on chip, update $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \mathbf{dS}_i^{(j)}\mathbf{K}_j \in \mathbb{R}^{B_r \times d}$, and write back to HBM.

    a. 在seq维度并行，并单独增加一个SM处理不同的column blocks更新dQ操作
5. 前向传播：每个worker处理注意力矩阵的一行；反向传播：每个worker处理注意力矩阵的一列



将工作分配到不同的warps：

1. FlashAttention-1 的做法 (Split-K)：在一个线程块内部，算法将 K 和 V 矩阵分割给不同的 Warp，而 Q 对所有 Warp 共享。
    a. 每个 Warp 计算出局部的注意力结果后，必须将其写入共享内存进行同步，然后加总。这种频繁的读写和同步操作显著降低了前向传播的速度
    b. 每个 Warp 计算出的实际上是输出矩阵 O 在列维度上的一个“片段”（即部分累加和）；由于多个 Warp 都在为同一行输出做贡献，它们必须把各自算出的“局部和”写到共享内存中，通过同步确保所有都算完了，最后再加在一起

2. FlashAttention-2 的做法 (Split-Q)：改为将 Q 矩阵分割给不同的 Warp（通常每块 4 或 8 个 Warp），而 K 和 V 则对所有 Warp 可见。
    a. 每个 Warp 在计算完其负责的 Q 片段与共享 K 的乘法后，直接与 V 结合即可得到对应的输出切片。这使得 Warp 之间完全不需要通信或同步，大幅减少了共享内存的访问
    b. 所有的 Warp 都能看到完整的 K 和 V 分块，但它们分别负责处理不同的 Query 行；由于每个 Warp 负责的是不同的输出行，它们之间没有任何数据重叠。Warp 1 算出的结果就是最终结果的一部分，不需要和 Warp 2 的结果相加
3. Block Size：
    a. 增加分块尺寸可以减少 HBM 加载次数，但会增加寄存器压力和共享内存占用
    b. 通常根据头维度 d 和 GPU 硬件资源，选择 {64,128}×{64,128} 大小的分块，以防止寄存器溢出导致的性能下降

总结：

FlashAttention 2 的速度比 FlashAttention 快约 2 倍，前向传播吞吐量最高可达理论最大吞吐量的 73%，反向传播吞吐量最高可达理论最大吞吐量的 63%。

Reference：

FlashAttention
FlashAttention2